

Project—矩阵乘法运算单元设计思路介绍

傅子酌 fuzizhuo@stu.pku.edu.cn

1 关于 systolic 阵列、broadcast 阵列和加法树的分析

人工智能应用中广泛使用张量计算，尤其是大型矩阵的乘法累加运算，是深度神经网络推理和训练的核心。我们以比较通用的矩阵乘加运算 $A \times B + C$ 为例来进行介绍（本次 project 中没有 C 矩阵）。图1展示了相应的矩阵运算操作，在神经网络中，矩阵 A 可表示输入激活值，矩阵 B 表示权重，矩阵 C 则为偏置项，计算得到的结果矩阵 D 即为输出。

$$D(m \times n) = A(m \times k) \times B(k \times n) + C(m \times n)$$

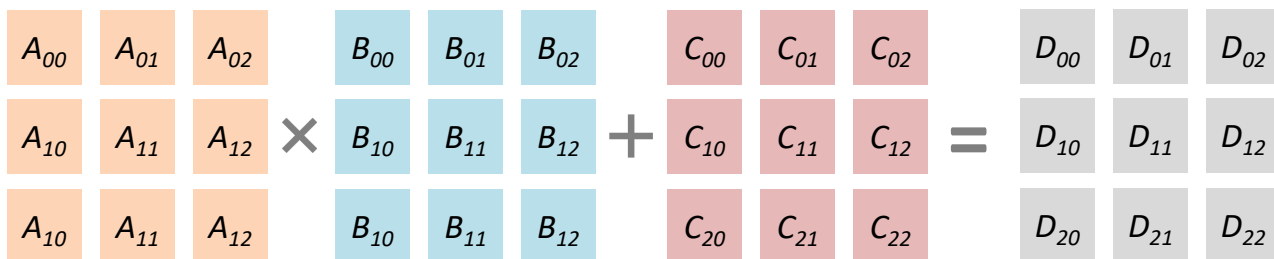


图1 矩阵乘加运算示意图：矩阵 A 与 B 相乘，加上矩阵 C 得到结果矩阵 D

矩阵乘法加速硬件常采用阵列结构，其中以脉动阵列（systolic array）和广播阵列（broadcast array）两种架构为代表，同时也有直接采用树形加法器（adder tree）实现的方法。图2是这三种方法的示意图。

脉动阵列：脉动阵列通过将数据在阵列单元间依次传递，实现高效的局部数据复用，包括固定输出（output stationary）、固定权重（weight stationary）、固定输入（input stationary）三种实现方式。图2(a)中，展示了 output stationary 的方法，从图中可以看出脉动阵列的优点在于连线简洁，架构灵活，但缺点在于需要进行数据重排，同时硬件资源利用率低，在矩阵运算的初始和结束阶段部分 PE 空闲，相应地导致运算所需周期数更多，当然，如果矩阵的中间维度较大，相应的开始

和结束时的空闲时间可忽略。

$$D(m \times n) = A(m \times k) \times B(k \times n) + C(m \times n)$$

Example: 3×3 GEMM, 9 Multiplier, 9 Adder

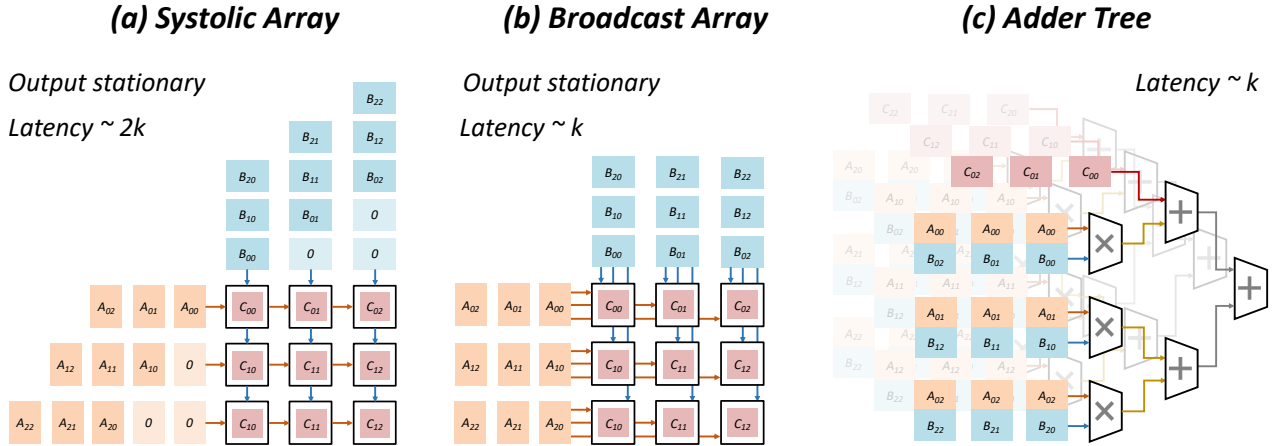


图 2 脉动阵列、广播阵列和加法树的方法比较，其中控制使用的硬件资源相同。在矩阵乘法的不同粒度上，可以采用不同的实现方式。

广播阵列： 广播阵列将需要运算的数据广播给多个 PE，同时进行并行计算。相比之下，广播结构可以在每个时钟周期向所有 PE 同时提供新的输入数据，令各 PE 并行计算当前部分乘积并累加到各自的输出寄存器中（Output stationary）。图2(b)中，展示了广播阵列的设计实现，其优点在于硬件资源利用率高，运算所需周期数较少，且架构灵活，相比于脉动阵列无需数据重排，但缺点在于布局布线更加复杂，当采用的阵列较大时，会导致更大的面积和能耗。

加法树： 加法树架构将所有乘法器的输出首先并行计算，再通过多级二叉树形加法网络完成累加。每一级加法器将来自下一级或乘法单元的若干部分积两两相加，直至根节点处输出最终结果，如图2(c)所示。由于各级加法器可独立流水，硬件设计相对简单，只需复用标准加法单元即可；同时，树形结构具有很高的模块化灵活性，能够根据累加项数动态调整网络深度，硬件资源利用率亦较高。然而，在顶层模块（Top Module）中，需要对多级流水级和数据到达时序进行精细调度，保证各级加法器的输入在正确的时钟周期到达，并协调乘法器与加法器之间的数据依赖；否则易出现气泡或数据冲突，从而影响整体吞吐性能。

以上三种方法实际上可以在不同的粒度任意结合使用，下面给出两个示例。

2 矩阵乘法加速器设计示例

2.1 示例 1：广播阵列与加法树相结合

图3给出了一个多精度 8×8 广播阵列矩阵乘法加速器的整体架构示意。阵列由 8 行 8 列共 64 个 PE 组成，每个 PE 对应输出矩阵中的一个元素位置，采用 output stationary 方式累加其对应的部分和。PE 的整体输入输出位宽是 64 位，对于 A、B 矩阵单边各 32 位。

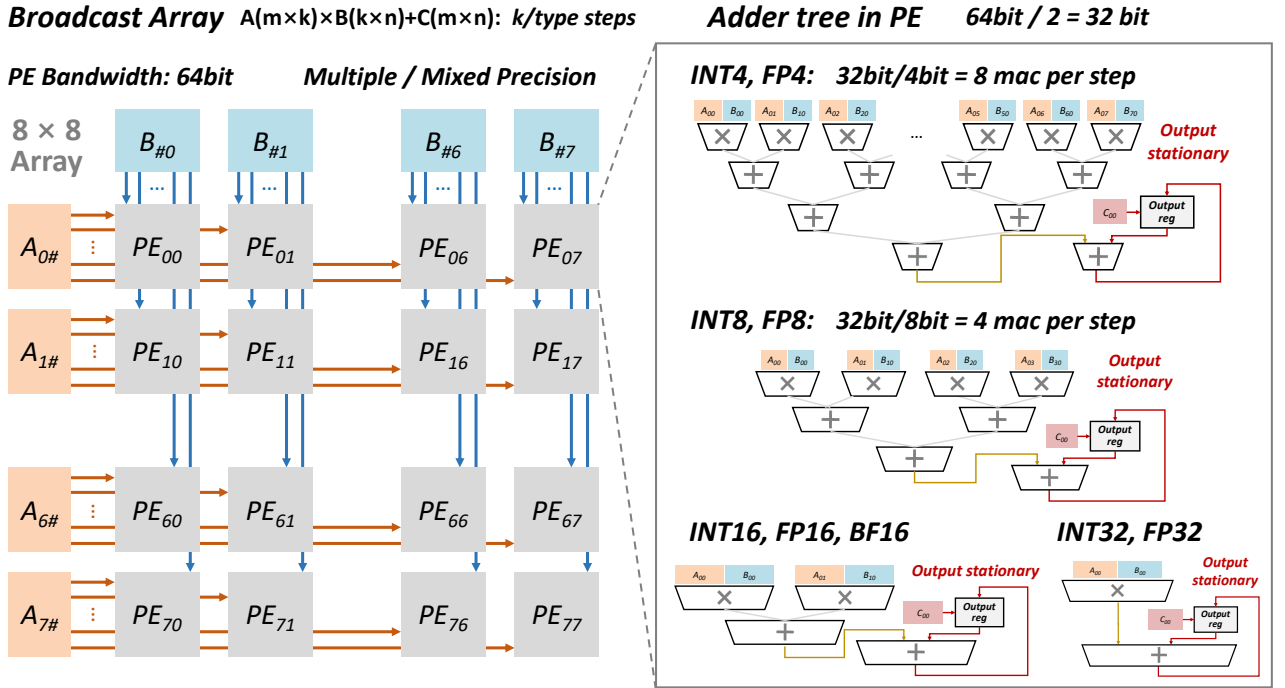


图 3 8×8 广播阵列整体架构：矩阵 A 各行数据广播到每行 PE，矩阵 B 各列数据广播到每列 PE。每个 PE 接收对应行的 A 数据和对应列的 B 数据，计算乘积累加到本 PE 的输出寄存器。

具体运算中首先对于矩阵 A 和矩阵 B 采用分块矩阵相乘的方法，分别在行和列的维度上按照 8 的粒度进行拆分，将原有任务 $D(m \times n) = A(m \times k) \times B(k \times n) + C(m \times n)$ ，拆分为多个 $D_{ij}(8 \times 8) = A_i(8 \times k) \times B_j(k \times 8) + C_{ij}(8 \times 8)$ 的子任务，如图4所示，以 $m = 16, k = 16, n = 16$ 为例。

现在每一个子任务中都是实现 $D_{ij}(8 \times 8)$ 维度的矩阵乘法，直接采用上述的 8×8 广播阵列进行运算。矩阵 A 的数据按行拆分，再逐行送入阵列的行输入端（如图中橙色箭头所示 A0# 至 A7# 分别代表第 0 至 7 行的当前输入）。矩阵 B 的数据按列存放，逐列送入阵列各列的输入端（蓝色箭头 B#0 至 B#7 表示第

Example: $m = 16, k = 16, n = 16 \rightarrow m = 8, k = 16, n = 8 (\times 4)$ Block matrix multiplication

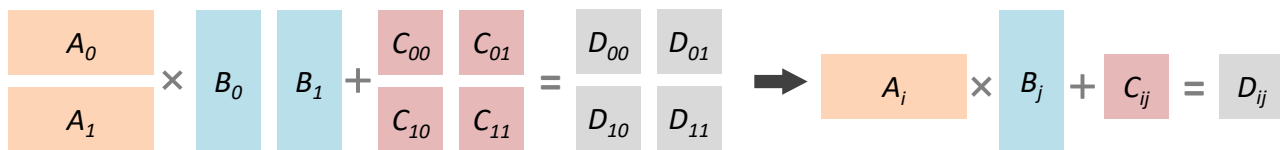


图 4 通过分块矩阵乘法实现高维度矩阵运算

0 至 7 列当前输入)。在计算开始前，还将矩阵 $C_{ij}(8 \times 8)$ (与输出矩阵同维度) 预先加载到各 PE 的输出寄存器中，作为初始累加值。

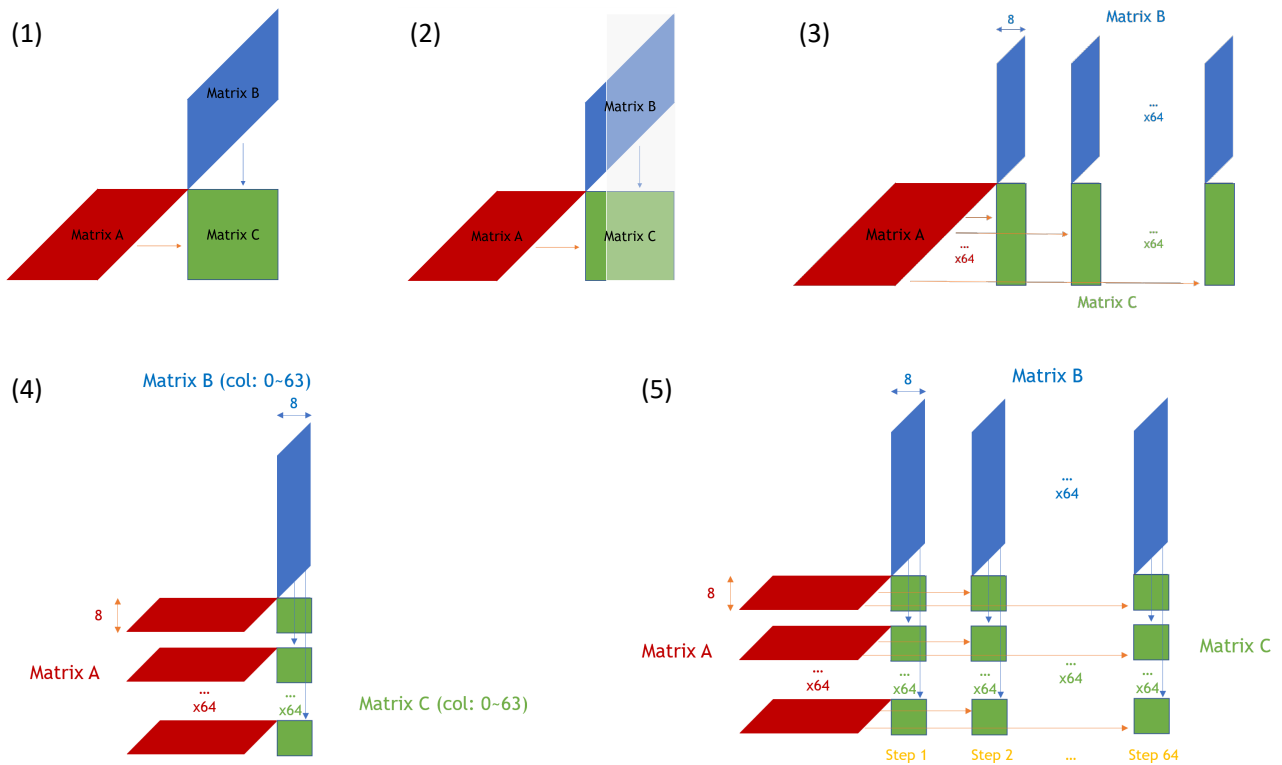
由于 PE 单边输入位宽是 32 位，所以对于 INT4 每周期可以输入 8 个数据，对于 INT8 每周期可以输入 4 个数据，对于 FP16 每周期可以输入 2 个数据，对于 FP32 每周期可以输入 1 个数据。针对不同数据类型以及位宽，单周期会有不同的数据输入数目，相应的在 PE 内部需要采用不同的加法器树形结构，如图3所示。通过这个多精度矩阵乘法设计可以看到，在每一个 PE 内部可以采用多种结构，这其实与每个 PE 的输入数据位宽相关。本次 project 中不考虑多精度，但关于位宽相关的设计仍需要仔细思考。

2.2 示例 2：广播阵列与脉动阵列相结合

下面给出一个 512×512 的矩阵乘法加速器示例，与本次 project 实现要求一致。但请注意：不同学期设计目标并不相同，侧重于不同的优化方向，本实现仅作为设计流程和思路的参考!!! 请自行完成设计过程!!! 同样地，采用往届同学的实现不一定会取得好的效果。

整体计算步骤分为 4 步，输入数据、计算结果、结果暂存、输出结果，为了降低延迟，4 个步骤要尽量并行执行，能够 overlap。本次实验中，整体分为两个部分，第一个部分输入 A 矩阵，将 A 矩阵存储到 SRAM 中，共 32768 个时钟周期。第二个部分同时进行：输入 B 矩阵，计算结果，结果暂存，输出结果，共 68112 个时钟周期。共计 100880 个时钟周期完成从开始输入到结束输出。

普通 512×512 矩阵脉动阵列运算方式 如图 5(1) 所示，可以直接使用 512×512 的大脉动阵列进行计算。但一方面，面积过大，另一方面，由于位宽限制，这样的计算基本要等到两个矩阵均输入完毕后才能开始计算，导致 memory-bound，计算资

图 5 512×512 矩阵乘法架构设计

源利用率极低，输出同理。

按列分块运算 本方案中，A 矩阵提前输入并存储，但 B 矩阵计算时输入，一次仅能拿到 8 个数据，所以按列划分来分块计算，得到每 8 列的结果，如图 5(2) 所示。采用按列分块运算的方式，最终的计算方式如图 5(3) 所示，分 64 步计算 64 个 512×8 分块矩阵的结果。

在按列的基础上，按行分块运算 上面按列分块运算的每一步都是得到一个 512×8 的子矩阵，但是需要等待 1024 个时钟周期才能输入，为了减小延迟，对于每一列的运算，按行分为 64 个 8×8 的子矩阵直接运算，如图 5(4) 所示。这实质上就是在不同粒度上采用了不同的阵列架构： $512 \times 512 = 64 \times (64 \times 1) \times (8 \times 8)$ ，其中，64 代表的是分为 64 个时间步骤， (64×1) 代表的是在这个粒度上采用 64×1 的广播阵列， (8×8) 代表的是在这个粒度上采用 8×8 的脉动阵列

计算流程 具体运算流程可参考图 6，在设计过程中要考虑是否能够 pipeline，以最大化带宽和计算资源的利用。

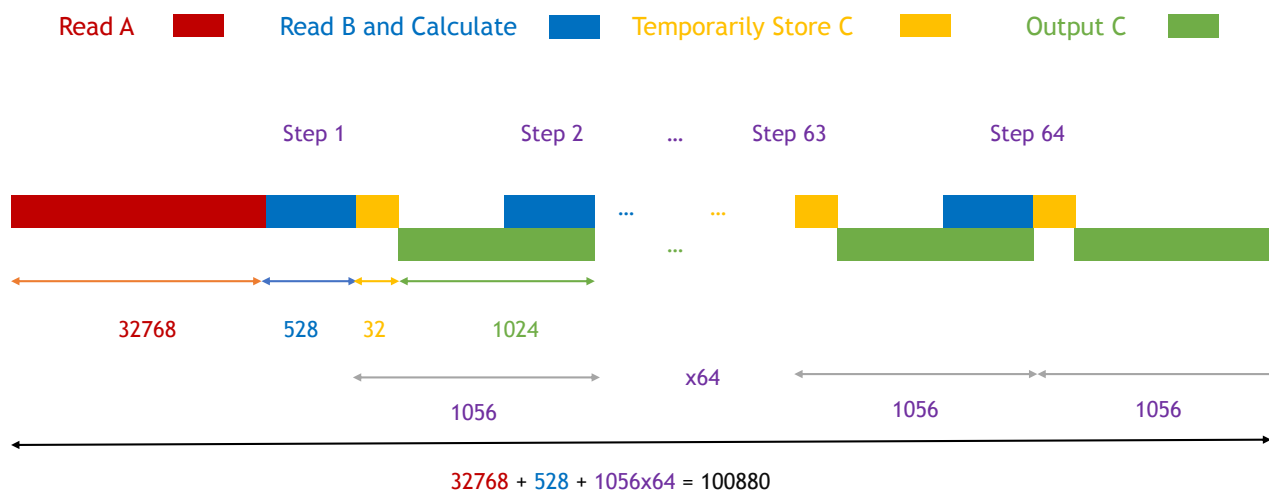


图 6 整体计算流程

3 设计建议

- 根据设计目标，提前确定整体设计需求。硬件设计的关键是 **trade-off**，在选择设计方案时，往往会面临类似“降低一倍延迟，增大一倍面积”，“降低一倍面积，增大一倍延迟”这样的选择，这时候需要根据设计目标来做抉择。
- 多考虑是否能够 **overlap**，是否能够进一步增大硬件资源利用率。
- 在确定设计之后，实际上延迟的周期数目也应该相应的确定了。同时，可以综合一些较小的模块（例如乘法器、加法器、单个 PE），对于各个部分的面积功耗有初步的认知。
- 一定要层次化设计，只要能划分出不同模块，就用多个不同层次的模块来写，便于之后的更改和仿真验证。
- 硬件设计中很大一部分工作量都在于写 **testbench** 来进行仿真验证，一定要单独验证确保每个模块功能和预想一致。
- Verilog/System Verilog 的代码书写不建议过度依赖 AI 工具，硬件社区的开源程度较低，没有充分数据集训练的情况下，AI 的硬件编程能力较差。可以采用 Copilot/Cursor 等代码补全功能，或者生成较小的定义清楚的模块。（2026 年 6 月注：此建议已经是 2024 年时代的眼泪了，在芯片前后端设计都要被 AI 打穿的当下，积极拥抱 AI 吧）